Question 1

Solve the Water Jug Problem using AI Search
Problem Statement:
You are given two jugs:

- Jug A = **4 gallons**
- Jug B = **3 gallons**

Initially, both jugs are empty. Using only the pump, pouring water between jugs, and emptying jugs, obtain exactly **2 gallons in the 4-gallon jug**.

State Representation
Represent each state as:
(A, B)
Where:

- **A** = Amount of water in the 4-gallon jug
- **B** = Amount of water in the 3-gallon jug

Initial State:
(0,0)
Goal State:
(2,x) where **x** can be 0, 1, 2, or 3.

Operators
The possible operations are:

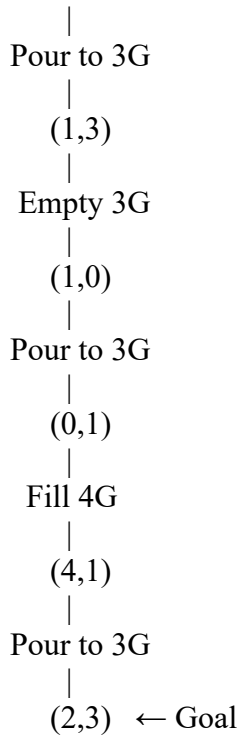
1. Fill the 4-gallon jug.
2. Fill the 3-gallon jug.
3. Empty the 4-gallon jug.
4. Empty the 3-gallon jug.
5. Pour water from the 4-gallon jug to the 3-gallon jug.
6. Pour water from the 3-gallon jug to the 4-gallon jug.

State Space Solution

Step	State (4G,3G)	Operation
1	(0,0)	Initial State
2	(4,0)	Fill 4-gallon jug
3	(1,3)	Pour 4G → 3G
4	(1,0)	Empty 3-gallon jug
5	(0,1)	Pour remaining 1 gallon into 4G
6	(4,1)	Fill 4-gallon jug
7	**(2,3)**	Pour 4G → 3G until full

Goal Achieved:
The **4-gallon jug contains exactly 2 gallons.**

State Space Diagram



Search Strategy
The Water Jug problem is a **state-space search problem**. Each state represents the amount of water in both jugs. AI explores possible states using search algorithms such as **Breadth-First Search (BFS)** or **Depth-First Search (DFS)** until the goal state is reached. BFS is generally preferred because it guarantees the shortest solution path.

Algorithm (BFS Approach)
1. Start from the initial state **(0,0)**.
2. Generate all possible next states using the allowed operations.
3. Avoid revisiting previously explored states.
4. Continue exploring level by level.
5. Stop when the state **(2,x)** is reached.

Applications
- AI problem solving
- Robotics
- State-space search
- Puzzle solving
- Planning and scheduling

Result
Using the state-space search approach, the Water Jug problem is solved successfully. The goal state **(2,3)** is reached, where the **4-gallon jug contains exactly 2 gallons**, satisfying the given condition.

Question 2

Mars Rover – Intelligent Agent Analysis
i. What are the percepts for this agent?

Percepts are the information collected by the Mars Rover through its sensors.

- Images captured by onboard cameras
- Rock and soil composition
- Atmospheric temperature and pressure
- Terrain and obstacle information
- Radiation levels
- Position and GPS/navigation data
- Wheel status and battery level

ii. Characterize the operating environment

Property	**Description**
Observability	Partially Observable
Agents	Single Agent
Deterministic	Mostly Stochastic
Episodic/Sequential	Sequential
Static/Dynamic	Dynamic
Discrete/Continuous	Continuous
Known/Unknown	Partially Known

iii. Actions performed by the agent

- Move forward or backward
- Turn left or right
- Capture images
- Collect rock and soil samples
- Drill the surface
- Analyze collected samples
- Send data to Earth
- Avoid obstacles
- Recharge using solar panels

iv. Performance Measure

The Mars Rover is evaluated based on:

- Accuracy of sample collection
- Successful navigation
- Obstacle avoidance
- Battery efficiency
- Scientific discoveries
- Communication success
- Mission completion

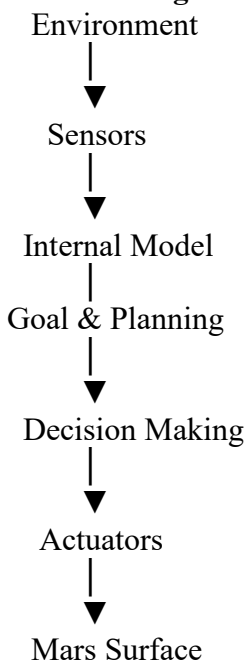
v. Suitable Agent Architecture

Model-Based Goal-Based Agent

Reason:

A Mars Rover operates in an uncertain environment. It stores information about previously visited locations, plans its route toward scientific goals, avoids obstacles, and makes intelligent decisions without continuous human control. Therefore, a **Model-Based Goal-Based Agent** is the most suitable architecture.

Architecture Diagram



Result

The Mars Rover is an intelligent autonomous agent that senses its environment, makes decisions, and performs scientific exploration using a **Model-Based Goal-Based Agent Architecture**.

Question 3

8-Queens Problem

Problem Formulation

The objective is to place **8 queens** on an **8 × 8 chessboard** such that no queen attacks another queen.

A queen can attack:

- Horizontally
- Vertically
- Diagonally

Problem Components

Initial State

Empty chessboard.

State

A partially filled chessboard with queens placed safely.

Operators

- Place one queen in a safe position.
- Move to the next column.

Goal State

Eight queens are placed without attacking each other.

Path Cost

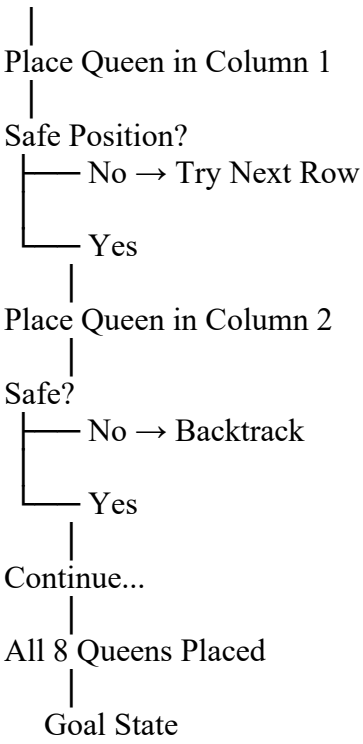
Each queen placement has equal cost.

Search Strategy

The 8-Queens problem is solved using **Backtracking Search**.
Steps:

1. Place the first queen in the first column.
2. Move to the next column.
3. Check whether the position is safe.
4. If safe, place the queen.
5. If not safe, try another row.
6. If no position is possible, backtrack.
7. Continue until all eight queens are placed.

Search Space Diagram



Example Solution

```
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
```

Advantages

- Reduces unnecessary searching.
- Uses backtracking to eliminate invalid states.
- Guarantees a valid solution if one exists.
- Efficient compared to brute-force search.

Applications

- AI search problems
- Scheduling systems
- Resource allocation
- Puzzle solving
- Constraint Satisfaction Problems (CSP)

Result
The **8-Queens Problem** is a **Constraint Satisfaction Problem (CSP)** solved efficiently using **Backtracking Search**, ensuring that all eight queens are placed without attacking each other.

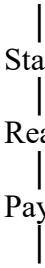
Question 4

OLA Cab Booking – Problem Solving Agent
Problem Statement
A customer wants to travel from one location to another using the **OLA Cab Booking** application. The customer can choose different cab types such as **Mini, Micro, Sedan, Shared, Prime**, etc., based on comfort and fare.

Type of Problem-Solving Agent
The most suitable AI agent is a **Goal-Based Agent**.
Reason
A Goal-Based Agent selects actions based on a specific goal. Here, the goal is to **reach the destination safely, quickly, and at minimum cost**. The agent evaluates different cab options and selects the one that best satisfies the user's requirements such as price, comfort, waiting time, and availability.

Working of the Agent
1. User enters pickup and destination locations.
2. The system calculates the available routes.
3. Available cab types are displayed.
4. The user selects a preferred cab.
5. The system calculates fare and estimated arrival time.
6. The nearest driver is assigned.
7. The trip begins and navigation is provided.
8. The customer reaches the destination and completes payment.

Flowchart
Start
|
Enter Pickup & Destination
|
Search Available Cabs
|
Display Cab Types
|
Select Preferred Cab
|
Calculate Fare & ETA
|
Assign Driver



Pseudocode
BEGIN

Input Pickup Location
Input Destination

Search Available Cabs

Display Cab Types

User Selects Cab

Calculate Fare

Assign Nearest Driver

Navigate Driver to Pickup

Start Trip

Reach Destination

Generate Bill

Complete Payment

END

Advantages
- Quick cab allocation
- Shortest route selection
- Reduced waiting time
- User-friendly booking process
- Real-time tracking
- Secure payment system

Result
A **Goal-Based Agent** is the most suitable AI agent for the OLA Cab Booking system because it selects the best cab option to achieve the customer's goal efficiently.

Question 5

Uniform Cost Search (UCS)
Problem Statement
Find the **least-cost path** from **S** (Start) to **G** (Goal) using the **Uniform Cost Search (UCS)** algorithm.

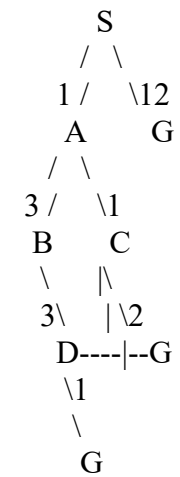
Edge Costs
From the graph:

- S → A = **1**
- S → G = **12**
- A → B = **3**
- A → C = **1**
- B → D = **3**
- C → D = **1**
- C → G = **2**
- D → G = **3**

Uniform Cost Search Steps

Step	Expanded Node	Path Cost
1	S	0
2	A	1
3	C	2
4	D	3
5	G	4

Search Tree



Cost Calculation
Path 1
S → G
Cost = **12**

Path 2
S → A → B → D → G
Cost

= 1 + 3 + 3 + 3
= **10**

Path 3
S → A → C → G
Cost
= 1 + 1 + 2
= **4**

Path 4
S → A → C → D → G
Cost
= 1 + 1 + 1 + 3
= **6**

Least Cost Path
S → A → C → G
Total Cost = 4

Working of UCS
Uniform Cost Search expands the node with the **lowest cumulative path cost** first. It uses a **priority queue** to store frontier nodes in increasing order of cost. The algorithm continues expanding nodes until the goal node is removed from the priority queue. UCS always guarantees the **optimal (least-cost) solution** when all edge costs are positive.

Advantages
- Finds the optimal solution.
- Works with different path costs.
- Complete and optimal algorithm.
- Suitable for route planning and navigation.
- Used in GPS, robotics, and logistics.

Result
Using the **Uniform Cost Search (UCS)** algorithm, the least-cost path is:
S → A → C → G
Minimum Cost = 4